

SQL Global Complete Reference

Goal: A practical, worldwide SQL reference covering standard SQL concepts plus the major SQL dialects and data-platform features.

Important: There is no finite list of literally "all SQL queries in the world." SQL is standardized, but every DBMS adds its own commands, functions, procedural language, extensions, and version-specific features. This document therefore covers the **major SQL language families, commands, query patterns, functions, window operations, partitioning, transactions, security, administration, and common vendor-specific syntax.**

Format: Topic → Query → When / Where to use

Dialect labels: ANSI/Standard , MySQL/MariaDB , PostgreSQL , SQL Server , Oracle , SQLite , BigQuery , Snowflake , Redshift , Databricks .

1. SQL Command Families

Family	Main purpose	Important commands
DQL	Read/query data	SELECT
DML	Change rows	INSERT , UPDATE , DELETE , MERGE
DDL	Define/change objects	CREATE , ALTER , DROP , TRUNCATE
DCL	Permissions/security	GRANT , REVOKE
TCL	Transactions	BEGIN , COMMIT , ROLLBACK , SAVEPOINT
Session/control	Session behavior	SET , USE , RESET , SHOW
Procedural SQL	Server-side logic	Functions, procedures, triggers, loops, variables
Analytical SQL	Advanced analysis	CTEs, windows, grouping sets, pivots, recursive queries

Administrative SQL	Metadata/performance	EXPLAIN , catalog queries, statistics, maintenance
Data loading	Bulk movement	COPY , LOAD DATA , BULK INSERT , INSERT ... SELECT

2. Core SELECT Queries

Topic	Query	When / Where to use
SELECT	SELECT name, salary FROM employee;	Read selected columns.
SELECT *	SELECT * FROM employee;	Quickly inspect all columns; avoid unnecessarily using * in production APIs.
DISTINCT	SELECT DISTINCT department FROM employee;	Return unique values.
WHERE	SELECT * FROM employee WHERE salary > 50000;	Filter individual rows.
AND	WHERE salary > 50000 AND department = 'IT'	Require all conditions.
OR	WHERE department = 'IT' OR department = 'HR'	Accept either condition.
NOT	WHERE NOT status = 'Inactive'	Negate a condition.
IN	WHERE department IN ('IT', 'HR', 'Sales')	Match a list of values.
NOT IN	WHERE department NOT IN ('HR', 'Finance')	Exclude listed values; watch for NULL semantics.
BETWEEN	WHERE salary BETWEEN 50000 AND 80000	Inclusive range filtering.
LIKE	WHERE name LIKE 'A%'	Pattern matching.

NOT LIKE	<code>WHERE name NOT LIKE 'A%'</code>	Exclude matching patterns.
IS NULL	<code>WHERE manager_id IS NULL</code>	Find missing/unknown values.
IS NOT NULL	<code>WHERE manager_id IS NOT NULL</code>	Find populated values.
ORDER BY	<code>SELECT * FROM employee ORDER BY salary DESC;</code>	Sort output.
ASC	<code>ORDER BY salary ASC</code>	Ascending sort.
DESC	<code>ORDER BY salary DESC</code>	Descending sort.
LIMIT	<code>SELECT * FROM employee LIMIT 10;</code>	MySQL/PostgreSQL/SQLite-style row limit.
OFFSET	<code>SELECT * FROM employee LIMIT 10 OFFSET 20;</code>	Offset-based pagination.
FETCH FIRST	<code>FETCH FIRST 10 ROWS ONLY</code>	Standard-style row limiting where supported.
TOP	<code>SELECT TOP 10 * FROM employee;</code>	SQL Server/Access row limiting.
WITH TIES	<code>SELECT TOP 10 WITH TIES * FROM employee ORDER BY salary DESC;</code>	Include rows tied at the boundary in SQL Server-style syntax.
Alias	<code>SELECT salary AS annual_salary FROM employee;</code>	Rename output columns.
Table alias	<code>FROM employee AS e</code>	Shorten table references and enable self-joins.
CASE	<code>CASE WHEN salary >= 80000 THEN 'High' ELSE 'Normal' END</code>	Conditional output.
COALESCE	<code>COALESCE(phone, email, 'No contact')</code>	Return the first non-NULL expression.
NULLIF	<code>amount / NULLIF(quantity, 0)</code>	Return NULL for equal values; commonly prevents division by zero.

3. INSERT / UPDATE / DELETE / MERGE

Topic	Query	When / Where to use
INSERT one row	<code>INSERT INTO employee(emp_id,name,salary) VALUES (1,'Yash',70000);</code>	Add one record.
INSERT many rows	<code>INSERT INTO employee(emp_id,name) VALUES (1,'A'),(2,'B');</code>	Add multiple records.
INSERT column list	<code>INSERT INTO employee(name,salary) VALUES ('A',70000);</code>	Prefer explicit column lists.
INSERT ... SELECT	<code>INSERT INTO archive SELECT * FROM employee WHERE status='Inactive';</code>	Copy query results into another table.
UPDATE	<code>UPDATE employee SET salary=75000 WHERE emp_id=1;</code>	Modify rows.
UPDATE multiple columns	<code>UPDATE employee SET salary=75000, city='Pune' WHERE emp_id=1;</code>	Change multiple columns.
Conditional UPDATE	<code>UPDATE employee SET salary = CASE WHEN dept_id=1 THEN salary*1.1 ELSE salary END;</code>	Apply different rules to different rows.
DELETE	<code>DELETE FROM employee WHERE emp_id=1;</code>	Delete selected rows.
DELETE all	<code>DELETE FROM employee;</code>	Remove all rows while retaining table structure.
MERGE	<code>MERGE INTO target USING source ON target.id=source.id ...;</code>	Synchronize source and target datasets.
PostgreSQL UPSERT	<code>INSERT ... ON CONFLICT (id) DO UPDATE SET ...;</code>	Insert or update on conflict.
MySQL/MariaDB UPSERT	<code>INSERT ... ON DUPLICATE KEY UPDATE ...;</code>	Insert or update on duplicate key.
SQLite UPSERT	<code>INSERT ... ON CONFLICT(id) DO UPDATE SET ...;</code>	SQLite insert-or-update pattern.

Oracle MERGE	<code>MERGE INTO target USING source ON (...) WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...;</code>	Oracle synchronization/upsert.
--------------	--	--------------------------------

4. DDL — DATABASE AND TABLE DEFINITION

Topic	Query	When / Where to use
CREATE DATABASE	<code>CREATE DATABASE company_db;</code>	Create a database where supported.
DROP DATABASE	<code>DROP DATABASE company_db;</code>	Delete a database and its objects.
CREATE SCHEMA	<code>CREATE SCHEMA sales;</code>	Create a namespace/schema.
DROP SCHEMA	<code>DROP SCHEMA sales;</code>	Remove a schema; cascade behavior is DBMS-specific.
CREATE TABLE	<code>CREATE TABLE employee (emp_id INT PRIMARY KEY, name VARCHAR(100));</code>	Create a table.
CREATE TABLE IF NOT EXISTS	<code>CREATE TABLE IF NOT EXISTS employee (...);</code>	Avoid an error if the table already exists where supported.
CREATE TABLE AS SELECT	<code>CREATE TABLE backup AS SELECT * FROM employee;</code>	Create a table from query output.
CREATE TEMP TABLE	<code>CREATE TEMPORARY TABLE temp_employee AS SELECT * FROM employee;</code>	Store session/intermediate data.
ALTER TABLE ADD	<code>ALTER TABLE employee ADD email VARCHAR(150);</code>	Add a column/constraint.
ALTER TABLE DROP COLUMN	<code>ALTER TABLE employee DROP COLUMN email;</code>	Remove a column.
ALTER TABLE RENAME COLUMN	<code>ALTER TABLE employee RENAME COLUMN name TO full_name;</code>	Rename a column; syntax varies.

ALTER TABLE RENAME	<code>ALTER TABLE employee RENAME TO staff;</code>	Rename a table where supported.
ALTER COLUMN TYPE	<code>ALTER TABLE employee ALTER COLUMN salary TYPE NUMERIC(12,2);</code>	Change a data type; syntax varies.
DROP TABLE	<code>DROP TABLE employee;</code>	Delete a table and its data.
TRUNCATE TABLE	<code>TRUNCATE TABLE employee;</code>	Remove all rows while retaining table structure.
CREATE VIEW	<code>CREATE VIEW active_employee AS SELECT * FROM employee WHERE active=1;</code>	Create a reusable logical query.
CREATE MATERIALIZED VIEW	<code>CREATE MATERIALIZED VIEW sales_summary AS SELECT ...;</code>	Store query results physically for faster reads where supported.
CREATE INDEX	<code>CREATE INDEX idx_employee_dept ON employee(dept_id);</code>	Improve suitable lookups/joins/sorts.
CREATE UNIQUE INDEX	<code>CREATE UNIQUE INDEX idx_email ON employee(email);</code>	Enforce uniqueness and support lookup.
DROP INDEX	<code>DROP INDEX idx_employee_dept;</code>	Remove an index; syntax varies.

5. CONSTRAINTS

Constraint	Query	When / Where to use
PRIMARY KEY	<code>PRIMARY KEY (emp_id)</code>	Unique identity for each row.
Composite PRIMARY KEY	<code>PRIMARY KEY (student_id, course_id)</code>	Uniqueness depends on multiple columns.
FOREIGN KEY	<code>FOREIGN KEY (dept_id) REFERENCES department(dept_id)</code>	Maintain referential integrity.
UNIQUE	<code>email VARCHAR(200) UNIQUE</code>	Prevent duplicate values.
NOT NULL	<code>name VARCHAR(100) NOT NULL</code>	Require a value.

CHECK	<code>CHECK (salary >= 0)</code>	Enforce a business/data rule.
DEFAULT	<code>status VARCHAR(20) DEFAULT 'Active'</code>	Provide a value automatically.
ON DELETE CASCADE	<code>FOREIGN KEY (...) REFERENCES ... ON DELETE CASCADE</code>	Delete child rows with parent.
ON DELETE SET NULL	<code>... ON DELETE SET NULL</code>	Clear child reference when parent is deleted.
ON UPDATE CASCADE	<code>... ON UPDATE CASCADE</code>	Propagate key changes where supported.

6. JOINS

Join	Query	When / Where to use
INNER JOIN	<code>SELECT e.name,d.name FROM employee e JOIN department d ON e.dept_id=d.dept_id;</code>	Only matching rows.
LEFT JOIN	<code>SELECT e.name,d.name FROM employee e LEFT JOIN department d ON e.dept_id=d.dept_id;</code>	All left rows + matches.
RIGHT JOIN	<code>SELECT e.name,d.name FROM employee e RIGHT JOIN department d ON e.dept_id=d.dept_id;</code>	All right rows + matches.
FULL OUTER JOIN	<code>SELECT * FROM employee e FULL OUTER JOIN department d ON e.dept_id=d.dept_id;</code>	All rows from both sides; support varies.
CROSS JOIN	<code>SELECT * FROM employee CROSS JOIN department;</code>	Every possible combination.
SELF JOIN	<code>SELECT e.name,m.name FROM employee e JOIN employee m ON e.manager_id=m.emp_id;</code>	Hierarchies such as employee/manager.

JOIN ... USING	<code>SELECT * FROM employee JOIN department USING (dept_id);</code>	Convenient when both columns have the same name.
NATURAL JOIN	<code>SELECT * FROM employee NATURAL JOIN department;</code>	Automatically joins same-named columns; avoid in production when explicit joins are safer.
Anti-join with NOT EXISTS	<code>SELECT d.* FROM department d WHERE NOT EXISTS (SELECT 1 FROM employee e WHERE e.dept_id=d.dept_id);</code>	Find parent rows without children.
Anti-join with LEFT JOIN	<code>SELECT d.* FROM department d LEFT JOIN employee e ON e.dept_id=d.dept_id WHERE e.emp_id IS NULL;</code>	Alternative no-match pattern.

7. AGGREGATION AND GROUPING

Topic	Query	When / Where to use
COUNT(*)	<code>SELECT COUNT(*) FROM employee;</code>	Count rows.
COUNT(column)	<code>SELECT COUNT(manager_id) FROM employee;</code>	Count non-NULL values.
COUNT(DISTINCT)	<code>SELECT COUNT(DISTINCT dept_id) FROM employee;</code>	Count unique values.
SUM	<code>SELECT SUM(salary) FROM employee;</code>	Calculate totals.
AVG	<code>SELECT AVG(salary) FROM employee;</code>	Calculate averages.
MIN	<code>SELECT MIN(salary) FROM employee;</code>	Find minimum.
MAX	<code>SELECT MAX(salary) FROM employee;</code>	Find maximum.

GROUP BY	<code>SELECT dept_id, AVG(salary) FROM employee GROUP BY dept_id;</code>	Aggregate per group.
HAVING	<code>... GROUP BY dept_id HAVING AVG(salary)>60000;</code>	Filter groups after aggregation.
Conditional aggregation	<code>SUM(CASE WHEN salary>=70000 THEN 1 ELSE 0 END)</code>	Calculate conditional counts/sums.
GROUPING SETS	<code>GROUP BY GROUPING SETS ((dept_id),(city),())</code>	Produce several grouping levels in one query.
ROLLUP	<code>GROUP BY ROLLUP (year, month)</code>	Create hierarchical subtotals and grand totals.
CUBE	<code>GROUP BY CUBE (region, product)</code>	Create combinations of subtotals; support varies.
FILTER	<code>COUNT(*) FILTER (WHERE salary>70000)</code>	Conditional aggregation in PostgreSQL/standard-style implementations.

8. SET OPERATORS

Operator	Query	When / Where to use
UNION	<code>SELECT city FROM customer UNION SELECT city FROM supplier;</code>	Combine compatible results and remove duplicates.
UNION ALL	<code>SELECT city FROM customer UNION ALL SELECT city FROM supplier;</code>	Combine results and retain duplicates.
INTERSECT	<code>SELECT city FROM customer INTERSECT SELECT city FROM supplier;</code>	Return rows present in both result sets.
EXCEPT	<code>SELECT city FROM customer EXCEPT SELECT city FROM supplier;</code>	Return rows in first result but not second.
MINUS	<code>SELECT city FROM customer MINUS SELECT city FROM supplier;</code>	Oracle equivalent of EXCEPT.

9. SUBQUERIES

Type	Query	When / Where to use
Scalar subquery	<code>WHERE salary > (SELECT AVG(salary) FROM employee)</code>	Use one returned value in another query.
IN subquery	<code>WHERE dept_id IN (SELECT dept_id FROM department WHERE region='West')</code>	Filter using a returned set.
EXISTS	<code>WHERE EXISTS (SELECT 1 FROM employee e WHERE e.dept_id=d.dept_id)</code>	Check whether related rows exist.
NOT EXISTS	<code>WHERE NOT EXISTS (...)</code>	Find rows with no related records.
Correlated subquery	<code>WHERE salary > (SELECT AVG(e2.salary) FROM employee e2 WHERE e2.dept_id=e.dept_id)</code>	Inner query depends on the outer row.
ANY/SOME	<code>salary > ANY (SELECT salary FROM employee WHERE dept_id=2)</code>	Condition must match at least one returned value.
ALL	<code>salary > ALL (SELECT salary FROM employee WHERE dept_id=2)</code>	Condition must match every returned value.
Derived table	<code>FROM (SELECT dept_id,AVG(salary) avg_sal FROM employee GROUP BY dept_id) x</code>	Use a query result as a table.

10. CTEs — COMMON TABLE EXPRESSIONS

CTE	Query	When / Where to use
Simple CTE	<code>WITH high_paid AS (SELECT * FROM employee WHERE salary>80000) SELECT * FROM high_paid;</code>	Break complex SQL into readable steps.
Multiple CTEs	<code>WITH a AS (...), b AS (...) SELECT ...;</code>	Build a multi-stage transformation.
Recursive CTE	<code>WITH RECURSIVE tree AS (...) SELECT * FROM tree;</code>	Hierarchies, trees, graph traversal, sequences.

Recursive sequence	<pre>WITH RECURSIVE n AS (SELECT 1 UNION ALL SELECT n+1 FROM n WHERE n<10) SELECT * FROM n;</pre>	Generate a sequence where recursive CTEs are supported.
--------------------	--	---

11. WINDOW FUNCTIONS — INCLUDING PARTITION BY

PARTITION BY is included here because it is one of the most important advanced SQL features. It is different from physical table partitioning.

Window feature	Query	When / Where to use
ROW_NUMBER	<pre>ROW_NUMBER() OVER (ORDER BY salary DESC)</pre>	Give every row a unique sequence.
RANK	<pre>RANK() OVER (ORDER BY salary DESC)</pre>	Rank with gaps after ties.
DENSE_RANK	<pre>DENSE_RANK() OVER (ORDER BY salary DESC)</pre>	Rank with no gaps after ties.
PARTITION BY	<pre>RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)</pre>	Perform a window calculation separately inside each group without collapsing rows.
ORDER BY in window	<pre>SUM(amount) OVER (ORDER BY order_date)</pre>	Define the order used by a window calculation.
LAG	<pre>LAG(salary) OVER (ORDER BY joining_date)</pre>	Read a previous row.
LEAD	<pre>LEAD(salary) OVER (ORDER BY joining_date)</pre>	Read a following row.
FIRST_VALUE	<pre>FIRST_VALUE(salary) OVER (PARTITION BY dept_id ORDER BY salary DESC)</pre>	Get the first value in a window.

LAST_VALUE	<code>LAST_VALUE(salary) OVER (ORDER BY salary ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)</code>	Get the final value in a specified frame.
NTH_VALUE	<code>NTH_VALUE(salary,2) OVER (ORDER BY salary DESC)</code>	Get the Nth value in a window where supported.
NTILE	<code>NTILE(4) OVER (ORDER BY salary DESC)</code>	Divide rows into buckets such as quartiles.
PERCENT_RANK	<code>PERCENT_RANK() OVER (ORDER BY salary)</code>	Calculate relative rank.
CUME_DIST	<code>CUME_DIST() OVER (ORDER BY salary)</code>	Calculate cumulative distribution.
Running SUM	<code>SUM(amount) OVER (ORDER BY order_date ROWS UNBOUNDED PRECEDING)</code>	Running/cumulative total.
Running AVG	<code>AVG(amount) OVER (ORDER BY order_date ROWS UNBOUNDED PRECEDING)</code>	Running average.
Moving average	<code>AVG(amount) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)</code>	Rolling 7-row average.
Window frame	<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>	Control which rows contribute to a window calculation.
Named WINDOW	<code>WINDOW w AS (PARTITION BY dept_id ORDER BY salary DESC)</code>	Reuse a window definition where supported.

12. TOP-N / RANKING PATTERNS

Pattern	Query / method	When / Where to use
Highest salary	<code>SELECT MAX(salary) FROM employee;</code>	Get the maximum value.

Employee with highest salary	<code>SELECT * FROM employee WHERE salary=(SELECT MAX(salary) FROM employee);</code>	Return the full row.
Second distinct salary	<code>DENSE_RANK() OVER (ORDER BY salary DESC)</code> then filter rank =2	Find the second distinct highest value.
Nth distinct salary	<code>DENSE_RANK() OVER (ORDER BY salary DESC)</code> then filter rank =N	General Nth-highest pattern.
Top 3 overall	<code>ORDER BY salary DESC LIMIT 3</code>	Return the top 3 rows.
Top 3 per department	<code>ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)</code> then <code>WHERE rn<=3</code>	Top N within every group.
Highest salary per department	<code>ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)</code> then <code>WHERE rn=1</code>	One top row per group.
All tied top salaries	<code>DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)</code> then <code>WHERE rn=1</code>	Keep ties.

13. STRING FUNCTIONS — COMMON SQL

Function	Query	When / Where to use
CONCAT	<code>CONCAT(first_name, ' ', last_name)</code>	Join strings.
CONCAT_WS	<code>CONCAT_WS(' ', city, state)</code>	Join strings with a separator.
UPPER	<code>UPPER(name)</code>	Uppercase normalization.
LOWER	<code>LOWER(email)</code>	Lowercase normalization.
TRIM	<code>TRIM(name)</code>	Remove leading/trailing spaces.
LTRIM	<code>LTRIM(name)</code>	Remove leading spaces.
RTRIM	<code>RTRIM(name)</code>	Remove trailing spaces.
LENGTH / LEN	<code>LENGTH(name) / LEN(name)</code>	Measure string length; name differs by DBMS.

SUBSTRING	<code>SUBSTRING(name, 1, 3)</code>	Extract text.
LEFT	<code>LEFT(name, 3)</code>	Take leftmost characters where supported.
RIGHT	<code>RIGHT(name, 3)</code>	Take rightmost characters where supported.
REPLACE	<code>REPLACE(city, 'Pune', 'PUNE')</code>	Replace text.
POSITION	<code>POSITION('a' IN name)</code>	Find substring position in standard-style syntax.
CHARINDEX	<code>CHARINDEX('a', name)</code>	SQL Server substring position.
INSTR	<code>INSTR(name, 'a')</code>	MySQL/Oracle-style substring position.
LPAD/RPAD	<code>LPAD(code, 5, '0')</code>	Pad values for formatting.
REVERSE	<code>REVERSE(name)</code>	Reverse text.
REPEAT / REPLICATE	<code>REPEAT('*', 10) / REPLICATE('*', 10)</code>	Repeat a string; name differs by DBMS.
STRING_AGG	<code>STRING_AGG(name, ',')</code>	Aggregate strings; PostgreSQL/SQL Server.
GROUP_CONCAT	<code>GROUP_CONCAT(name)</code>	MySQL/MariaDB string aggregation.
LISTAGG	<code>LISTAGG(name, ',') WITHIN GROUP (ORDER BY name)</code>	Oracle/warehouse string aggregation.

14. NUMERIC FUNCTIONS — COMMON SQL

Function	Query	When / Where to use
ABS	<code>ABS(x)</code>	Absolute value.
ROUND	<code>ROUND(x, 2)</code>	Round decimal values.
CEIL / CEILING	<code>CEILING(x)</code>	Round upward.

FLOOR	<code>FLOOR(x)</code>	Round downward.
TRUNC / TRUNCATE	<code>TRUNC(x, 2)</code> / <code>TRUNCATE(x, 2)</code>	Remove decimal precision without normal rounding; name varies.
MOD	<code>MOD(x, y)</code>	Remainder.
POWER / POW	<code>POWER(x, 2)</code>	Raise to a power.
SQRT	<code>SQRT(x)</code>	Square root.
SIGN	<code>SIGN(x)</code>	Negative/zero/positive sign.
EXP	<code>EXP(x)</code>	e raised to x.
LN	<code>LN(x)</code>	Natural logarithm where supported.
LOG / LOG10	<code>LOG10(x)</code>	Logarithms.
GREATEST	<code>GREATEST(a, b, c)</code>	Largest among expressions where supported.
LEAST	<code>LEAST(a, b, c)</code>	Smallest among expressions where supported.
RANDOM	<code>RANDOM()</code> / <code>RAND()</code>	Random values; function name differs.
PI	<code>PI()</code>	Mathematical constant pi.

15. DATE / TIME QUERIES

Date operation	Query	When / Where to use
Current date	<code>CURRENT_DATE</code>	Current date.
Current time	<code>CURRENT_TIME</code>	Current time.
Current timestamp	<code>CURRENT_TIMESTAMP</code>	Current date and time.
MySQL now	<code>NOW()</code>	Current MySQL date/time.

SQL Server now	<code>GETDATE()</code>	Current SQL Server date/time.
PostgreSQL now	<code>NOW()</code>	Current PostgreSQL timestamp.
Oracle now	<code>SYSDATE</code>	Current Oracle server date/time.
Extract year	<code>EXTRACT(YEAR FROM order_date)</code>	Get year.
Extract month	<code>EXTRACT(MONTH FROM order_date)</code>	Get month.
Extract day	<code>EXTRACT(DAY FROM order_date)</code>	Get day.
YEAR function	<code>YEAR(order_date)</code>	MySQL/SQL Server-style year extraction.
MONTH function	<code>MONTH(order_date)</code>	MySQL/SQL Server-style month extraction.
DAY function	<code>DAY(order_date)</code>	MySQL/SQL Server-style day extraction.
Date addition	<code>DATE_ADD(order_date, INTERVAL 7 DAY)</code>	MySQL add interval.
SQL Server date addition	<code>DATEADD(day, 7, order_date)</code>	SQL Server add interval.
PostgreSQL date addition	<code>order_date + INTERVAL '7 days'</code>	PostgreSQL interval arithmetic.
Date difference	<code>DATEDIFF(end_date, start_date)</code>	Difference; exact semantics vary.
PostgreSQL difference	<code>end_date - start_date</code>	Date difference in PostgreSQL.
Date formatting	<code>DATE_FORMAT(order_date, '%Y-%m-%d')</code>	MySQL display formatting.

SQL Server formatting	<code>FORMAT(order_date, 'yyyy-MM-dd')</code>	SQL Server display formatting.
Truncate to month	<code>DATE_TRUNC('month', order_date)</code>	PostgreSQL/Snowflake/Redshift/BigQuery-style period bucketing.
Half-open date range	<code>WHERE ts >= '2026-01-01' AND ts < '2027-01-01'</code>	Robust timestamp range filtering.

16. CASTING AND CONVERSION

Operation	Query	When / Where to use
CAST	<code>CAST(salary AS DECIMAL(12,2))</code>	Explicit type conversion; broadly standard.
CONVERT	<code>CONVERT(DECIMAL(12,2), salary)</code>	SQL Server/MySQL-specific conversion syntax.
PostgreSQL cast	<code>salary::NUMERIC(12,2)</code>	PostgreSQL shorthand.
TRY_CAST	<code>TRY_CAST(value AS INT)</code>	SQL Server/Snowflake-style safe conversion where supported; returns NULL instead of failing for invalid input.
SAFE_CAST	<code>SAFE_CAST(value AS INT64)</code>	BigQuery safe conversion.
PARSE_DATE	<code>PARSE_DATE('%Y-%m-%d', value)</code>	BigQuery-style parsing.
TO_DATE	<code>TO_DATE(value, 'YYYY-MM-DD')</code>	Oracle/PostgreSQL/Snowflake-style date parsing depending on dialect.

17. NULL AND THREE-VALUED LOGIC

Feature	Query	When / Where to use
IS NULL	<code>WHERE phone IS NULL</code>	Correct NULL test.

IS NOT NULL	<code>WHERE phone IS NOT NULL</code>	Find non-NULL values.
COALESCE	<code>COALESCE(phone, email, 'None')</code>	Fallback value.
NULLIF	<code>NULLIF(denominator, 0)</code>	Turn a problematic value into NULL.
MySQL IFNULL	<code>IFNULL(phone, 'None')</code>	MySQL/MariaDB two-value NULL fallback.
SQL Server ISNULL	<code>ISNULL(phone, 'None')</code>	SQL Server NULL replacement.
Oracle NVL	<code>NVL(phone, 'None')</code>	Oracle NULL replacement.
Oracle NVL2	<code>NVL2(phone, 'Has', 'Missing')</code>	Return one expression for non-NULL and another for NULL.

18. PHYSICAL TABLE PARTITIONING

This is **different from PARTITION BY inside a window function**. Physical partitioning divides table data for storage/management/performance. Exact syntax is highly DBMS-specific.

Partition type / DBMS	Query	When / Where to use
Range partitioning	<code>PARTITION BY RANGE (order_date)</code>	Split data into date/numeric ranges.
List partitioning	<code>PARTITION BY LIST (region)</code>	Split data by discrete categories.
Hash partitioning	<code>PARTITION BY HASH (customer_id)</code>	Distribute rows across partitions using a hash.
Composite partitioning	<code>PARTITION BY RANGE (...) SUBPARTITION BY HASH (...)</code>	Combine partitioning strategies where supported.
MySQL RANGE	<code>PARTITION BY RANGE (YEAR(order_date)) (...)</code>	Partition large tables by ranges.
MySQL LIST	<code>PARTITION BY LIST COLUMNS(region) (...)</code>	Partition by explicit values.

PostgreSQL range	<code>PARTITION BY RANGE (order_date)</code>	Create a partitioned parent table.
PostgreSQL list	<code>PARTITION BY LIST (region)</code>	Category-based partitions.
PostgreSQL hash	<code>PARTITION BY HASH (customer_id)</code>	Hash-distribute partitions.
SQL Server partition function	<code>CREATE PARTITION FUNCTION pf_date (...) AS RANGE RIGHT FOR VALUES (...);</code>	Define SQL Server partition boundaries.
SQL Server partition scheme	<code>CREATE PARTITION SCHEME ps_date AS PARTITION pf_date ALL TO ([PRIMARY]);</code>	Map SQL Server partitions to filegroups.
Oracle interval/range	<code>PARTITION BY RANGE (order_date) (...)</code>	Oracle table partitioning.
BigQuery partitioned table	<code>CREATE TABLE t PARTITION BY DATE(order_date) AS SELECT ...;</code>	Partition analytical tables for scan reduction.
Snowflake clustering	<code>CLUSTER BY (date_col)</code>	Snowflake commonly uses micro-partitioning automatically plus clustering keys for pruning; not the same syntax as traditional table partitions.

19. WINDOW PARTITIONING — PARTITION BY EXAMPLES

Pattern	Query	When / Where to use
Rank within department	<code>RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)</code>	Rank employees independently in each department.
Top employee per department	<code>ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)</code>	Filter <code>rn=1</code> .

Running total per customer	<code>SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)</code>	Cumulative customer spending.
Previous order per customer	<code>LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date)</code>	Find previous order/date.
Department average on every row	<code>AVG(salary) OVER (PARTITION BY dept_id)</code>	Compare each employee to department average without collapsing rows.
Salary percentage within department	<code>salary / SUM(salary) OVER (PARTITION BY dept_id)</code>	Calculate each employee's share.
Quartile per region	<code>NTILE(4) OVER (PARTITION BY region ORDER BY revenue DESC)</code>	Segment rows within each region.

20. WINDOW FRAME CLAUSES

Frame	Query	When / Where to use
ROWS preceding	<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>	Physical row-based moving window.
ROWS following	<code>ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING</code>	Current row plus future rows.
Full rows	<code>ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING</code>	Whole partition.
From beginning	<code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>	Running calculation.
RANGE	<code>RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>	Value-based frame semantics; behavior differs from ROWS when ties exist.
GROUPS	<code>GROUPS BETWEEN 1 PRECEDING AND CURRENT ROW</code>	Frame by peer groups where supported.

21. VIEWS AND MATERIALIZED VIEWS

Operation	Query	When / Where to use
CREATE VIEW	<code>CREATE VIEW active_employee AS SELECT ...;</code>	Reusable logical query.
CREATE OR REPLACE VIEW	<code>CREATE OR REPLACE VIEW active_employee AS SELECT ...;</code>	Replace a view where supported.
DROP VIEW	<code>DROP VIEW active_employee;</code>	Remove a view.
Materialized view	<code>CREATE MATERIALIZED VIEW summary AS SELECT ...;</code>	Persist query results for faster reads.
Refresh materialized view	<code>REFRESH MATERIALIZED VIEW summary;</code>	Refresh stored results where supported.

22. INDEXES AND PERFORMANCE

Feature	Query	When / Where to use
Single-column index	<code>CREATE INDEX idx_emp_dept ON employee(dept_id);</code>	Speed suitable filters/joins.
Composite index	<code>CREATE INDEX idx_emp_dept_salary ON employee(dept_id,salary);</code>	Support queries using columns together; order matters.
Unique index	<code>CREATE UNIQUE INDEX idx_email ON employee(email);</code>	Uniqueness + lookup.
Partial/filtered index	<code>CREATE INDEX ... WHERE active=true;</code>	Index only a subset where supported.
Expression/function index	<code>CREATE INDEX idx_lower_email ON users(LOWER(email));</code>	Index computed expressions where supported.
Covering/include index	<code>CREATE INDEX ... INCLUDE (name,salary);</code>	Store extra columns for index-only plans where supported.

Drop index	<code>DROP INDEX idx_emp_dept;</code>	Remove unnecessary indexes.
EXPLAIN	<code>EXPLAIN SELECT ...;</code>	Inspect planned execution.
EXPLAIN ANALYZE	<code>EXPLAIN ANALYZE SELECT ...;</code>	Inspect actual execution in supported DBMS.
Analyze statistics	<code>ANALYZE employee;</code>	Update optimizer statistics where supported.

23. TRANSACTIONS / TCL

Operation	Query	When / Where to use
BEGIN	<code>BEGIN;</code>	Start a transaction.
START TRANSACTION	<code>START TRANSACTION;</code>	Start a transaction in MySQL-style syntax.
COMMIT	<code>COMMIT;</code>	Make transaction changes durable.
ROLLBACK	<code>ROLLBACK;</code>	Undo uncommitted changes.
SAVEPOINT	<code>SAVEPOINT before_delete;</code>	Create a partial rollback point.
ROLLBACK TO SAVEPOINT	<code>ROLLBACK TO SAVEPOINT before_delete;</code>	Undo changes after a savepoint.
RELEASE SAVEPOINT	<code>RELEASE SAVEPOINT before_delete;</code>	Remove a savepoint where supported.
Read committed	<code>SET TRANSACTION ISOLATION LEVEL READ COMMITTED;</code>	Common isolation level.
Repeatable read	<code>SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;</code>	Stable reads within a transaction where supported.
Serializable	<code>SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;</code>	Strongest common isolation level; potentially lower concurrency.

Read uncommitted	SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;	Permit dirty reads where supported.
------------------	--	-------------------------------------

24. ROW LOCKING / CONCURRENCY

Lock	Query / pattern	When / Where to use
SELECT FOR UPDATE	SELECT * FROM seats WHERE id=10 FOR UPDATE;	Lock selected rows for update inside a transaction where supported.
SELECT FOR SHARE	SELECT * FROM item WHERE id=10 FOR SHARE;	Shared row lock in supporting DBMS.
SQL Server UPDLOCK	SELECT * FROM seats WITH (UPDLOCK) WHERE id=10;	Acquire update locks in SQL Server.
SQL Server HOLDLOCK	SELECT * FROM seats WITH (HOLDLOCK) WHERE id=10;	Serializable-like locking behavior for the table reference.
Advisory lock	DBMS-specific functions such as PostgreSQL advisory locks.	Coordinate application-level resources.

25. SECURITY / DCL

Security operation	Query	When / Where to use
CREATE ROLE	CREATE ROLE analyst;	Create a reusable security role where supported.
CREATE USER	CREATE USER app_user ...;	Create a database user; syntax varies.
GRANT SELECT	GRANT SELECT ON employee TO analyst;	Give read access.
GRANT multiple	GRANT SELECT, INSERT, UPDATE ON employee TO app_user;	Give several privileges.
GRANT ALL	GRANT ALL PRIVILEGES ON ... TO ...;	Grant broad permissions; use least privilege.

REVOKE	<code>REVOKE INSERT ON employee FROM analyst;</code>	Remove permissions.
DENY	<code>DENY SELECT ON employee TO user1;</code>	SQL Server explicit deny.
Role membership	<code>GRANT analyst TO app_user;</code>	Assign a role where supported.
Row-level security	<code>CREATE POLICY ...</code>	Restrict visible rows by user/role; PostgreSQL and other DBMS support different mechanisms.

26. METADATA / INFORMATION SCHEMA

Metadata operation	Query	When / Where to use
List tables	<code>SELECT table_name FROM information_schema.tables;</code>	Inspect portable-style metadata.
List columns	<code>SELECT table_name,column_name,data_type FROM information_schema.columns;</code>	Inspect schemas programmatically.
List constraints	<code>SELECT * FROM information_schema.table_constraints;</code>	Inspect constraints.
List key columns	<code>SELECT * FROM information_schema.key_column_usage;</code>	Inspect key relationships.
PostgreSQL catalog	<code>SELECT * FROM pg_catalog.pg_tables;</code>	PostgreSQL-specific metadata.
SQL Server catalog	<code>SELECT * FROM sys.tables;</code>	SQL Server metadata.
Oracle dictionary	<code>SELECT * FROM user_tables;</code>	Oracle metadata.
MySQL SHOW TABLES	<code>SHOW TABLES;</code>	MySQL/MariaDB table listing.

MySQL DESCRIBE	<code>DESCRIBE employee;</code>	MySQL/MariaDB table structure.
-------------------	---------------------------------	--------------------------------

27. TEMPORARY TABLES AND TABLE VARIABLES

Feature	Query	When / Where to use
Temporary table	<code>CREATE TEMPORARY TABLE tmp AS SELECT ...;</code>	Intermediate/session data.
SQL Server temp table	<code>CREATE TABLE #tmp (id INT);</code>	Session-scoped SQL Server temporary table.
SQL Server table variable	<code>DECLARE @t TABLE (id INT);</code>	Small/intermediate in-memory-style relational variable; optimizer behavior differs from temp tables.
PostgreSQL temporary	<code>CREATE TEMP TABLE tmp AS SELECT ...;</code>	Session/transaction temporary data.

28. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

Object	Query / pattern	When / Where to use
Stored procedure	<code>CREATE PROCEDURE ...</code>	Reusable server-side operations; syntax differs by DBMS.
Function	<code>CREATE FUNCTION ... RETURNS ...</code>	Reusable computation that returns a value/table depending on DBMS.
Trigger	<code>CREATE TRIGGER ...</code>	Automatically run logic on INSERT/UPDATE/DELETE events.
BEFORE trigger	<code>CREATE TRIGGER ... BEFORE INSERT ...</code>	Validate/transform before a row operation where supported.

AFTER trigger	<code>CREATE TRIGGER ... AFTER INSERT ...</code>	Audit/react after a row operation.
INSTEAD OF trigger	<code>CREATE TRIGGER ... INSTEAD OF ...</code>	Replace an operation, commonly for views in supporting DBMS.

29. PIVOT / UNPIVOT

Operation	Query / pattern	When / Where to use
Conditional pivot	<code>SUM(CASE WHEN month=1 THEN amount ELSE 0 END) AS Jan</code>	Portable way to turn categories into columns.
SQL Server PIVOT	<code>SELECT ... FROM (...) s PIVOT (SUM(amount) FOR month IN ([1],[2],[3])) p;</code>	Turn rows into columns.
SQL Server UNPIVOT	<code>SELECT ... FROM table UNPIVOT (amount FOR month IN (...)) u;</code>	Turn columns into rows.
BigQuery PIVOT	<code>SELECT * FROM t PIVOT(SUM(amount) FOR month IN (1,2,3));</code>	Analytical pivot where supported.
Snowflake PIVOT	<code>SELECT * FROM t PIVOT(SUM(amount) FOR month IN (1,2,3));</code>	Analytical pivot where supported.

30. JSON QUERIES

Feature	Query	When / Where to use
PostgreSQL JSON text	<code>SELECT data->'name' FROM users;</code>	Read a JSON field as text.
PostgreSQL JSON object	<code>SELECT data->'address' FROM users;</code>	Read nested JSON.
PostgreSQL JSON containment	<code>WHERE data @> '{"role": "admin"}'::jsonb</code>	Filter JSONB by contained structure.
MySQL JSON_EXTRACT	<code>SELECT JSON_EXTRACT(data, '\$.name') FROM users;</code>	Extract JSON values.

MySQL JSON path operator	<code>SELECT data->>'\$.name' FROM users;</code>	Convenient JSON scalar extraction where supported.
SQL Server JSON_VALUE	<code>SELECT JSON_VALUE(data, '\$.name') FROM users;</code>	Extract a scalar JSON value.
SQL Server OPENJSON	<code>SELECT * FROM OPENJSON(data);</code>	Turn JSON into relational rows.
BigQuery JSON_VALUE	<code>SELECT JSON_VALUE(data, '\$.name') FROM users;</code>	Extract JSON scalar.

31. ARRAY QUERIES

Feature	Query	When / Where to use
PostgreSQL ANY	<code>WHERE 'SQL' = ANY(tags)</code>	Check whether a value occurs in an array.
PostgreSQL array contains	<code>WHERE tags @> ARRAY['SQL']</code>	Test array containment.
UNNEST	<code>SELECT * FROM UNNEST(tags);</code>	Expand an array into rows; supported in several analytical DBMSs.
BigQuery UNNEST	<code>SELECT item FROM t, UNNEST(items) AS item;</code>	Flatten repeated/array data.

32. FULL-TEXT / TEXT SEARCH

Feature	Query	When / Where to use
PostgreSQL tsvector	<code>to_tsvector('english', body)</code>	Prepare text for PostgreSQL full-text search.
PostgreSQL tsquery	<code>to_tsquery('english', 'database & sql')</code>	Build a full-text search query.

PostgreSQL match	<code>WHERE to_tsvector(body) @@ plainto_tsquery('database')</code>	Search natural- language text.
MySQL FULLTEXT	<code>WHERE MATCH(title,body) AGAINST ('database' IN NATURAL LANGUAGE MODE)</code>	Full-text search with a FULLTEXT index.
SQL Server CONTAINS	<code>WHERE CONTAINS(body, '"database"')</code>	SQL Server full-text search.

33. RECURSIVE / HIERARCHICAL QUERIES

Pattern	Query	When / Where to use
Recursive CTE	<code>WITH RECURSIVE org AS (...) SELECT * FROM org;</code>	Employee trees, category trees, graph traversal.
Employee hierarchy	<code>WITH RECURSIVE org AS (SELECT ... UNION ALL SELECT ... FROM employee e JOIN org o ON e.manager_id=o.emp_id) SELECT * FROM org;</code>	Walk manager/employee hierarchy.
Generate numbers	<code>WITH RECURSIVE n AS (SELECT 1 UNION ALL SELECT n+1 FROM n WHERE n<100) SELECT * FROM n;</code>	Generate sequences.
Oracle CONNECT BY	<code>SELECT employee_id, manager_id FROM employee CONNECT BY PRIOR employee_id=manager_id START WITH manager_id IS NULL;</code>	Oracle hierarchical queries.

34. ANALYTICAL QUERY PATTERNS

Pattern	Query	When / Where to use
Running total	<code>SUM(amount) OVER (ORDER BY order_date)</code>	Cumulative metrics.
Running total per customer	<code>SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)</code>	Customer cumulative spend.
Previous value	<code>LAG(amount) OVER (ORDER BY order_date)</code>	Period-over-period comparison.

Next value	<code>LEAD(amount) OVER (ORDER BY order_date)</code>	Look ahead.
Growth rate	<code>(value - LAG(value) OVER (...)) / NULLIF(LAG(value) OVER (...),0)</code>	Calculate period growth.
Top N per group	<code>ROW_NUMBER() OVER (PARTITION BY group_id ORDER BY metric DESC)</code>	Top records in every category.
Duplicate detection	<code>COUNT(*) OVER (PARTITION BY email)</code>	Identify repeated values without collapsing rows.
Deduplicate	<code>ROW_NUMBER() OVER (PARTITION BY business_key ORDER BY id)</code>	Keep one row and identify extras.
Percent of total	<code>amount / SUM(amount) OVER ()</code>	Calculate share of total.
Percent of group	<code>amount / SUM(amount) OVER (PARTITION BY group_id)</code>	Calculate share inside a group.
Median	<code>PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)</code>	Calculate median where supported.
Percentile	<code>PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY value)</code>	Calculate a percentile where supported.

35. DATA QUALITY / CLEANING QUERIES

Quality task	Query	When / Where to use
Find duplicates	<code>SELECT email,COUNT(*) FROM users GROUP BY email HAVING COUNT(*)>1;</code>	Detect duplicate business keys.
Find NULLs	<code>SELECT COUNT(*) FROM users WHERE email IS NULL;</code>	Measure missing data.
Trim spaces	<code>UPDATE users SET name=TRIM(name);</code>	Clean whitespace.
Normalize case	<code>UPDATE users SET email=LOWER(TRIM(email));</code>	Normalize email/text values.
Invalid negative values	<code>SELECT * FROM product WHERE price<0;</code>	Detect impossible values.

Orphan records	<code>SELECT c.* FROM child c LEFT JOIN parent p ON p.id=c.parent_id WHERE p.id IS NULL;</code>	Detect broken relationships.
Duplicate-safe delete preview	<code>SELECT * FROM (...) x WHERE rn>1;</code>	Inspect duplicate candidates before deleting.

36. SAFE DELETE / UPDATE WORKFLOW

Pattern	Query	When / Where to use
Preview DELETE	<code>SELECT * FROM employee WHERE dept_id=4;</code>	Always inspect the target rows first.
Delete in transaction	<code>BEGIN; DELETE ...; SELECT ...; ROLLBACK;</code>	Test a destructive operation before committing.
Preview UPDATE	<code>SELECT emp_id,salary,salary*1.1 AS new_salary FROM employee WHERE dept_id=1;</code>	See the intended result before updating.
Update transaction	<code>BEGIN; UPDATE ...; SELECT ...; COMMIT;</code>	Protect important changes.

37. PAGINATION

Pattern	Query	When / Where to use
LIMIT/OFFSET	<code>SELECT * FROM employee ORDER BY emp_id LIMIT 20 OFFSET 40;</code>	Simple pagination.
TOP	<code>SELECT TOP 20 * FROM employee ORDER BY emp_id;</code>	SQL Server/Access-style first page.
FETCH/OFFSET	<code>OFFSET 40 ROWS FETCH NEXT 20 ROWS ONLY</code>	Standard/SQL Server/Oracle-style pagination.
Keyset pagination	<code>WHERE emp_id > :last_id ORDER BY emp_id LIMIT 20</code>	Efficient pagination for large indexed datasets.

38. UPSERT / DATA SYNCHRONIZATION

DBMS / pattern	Query	When / Where to use
PostgreSQL	<code>INSERT ... ON CONFLICT (...) DO UPDATE SET ...;</code>	Insert or update.
MySQL/MariaDB	<code>INSERT ... ON DUPLICATE KEY UPDATE ...;</code>	Insert or update.
SQLite	<code>INSERT ... ON CONFLICT (...) DO UPDATE SET ...;</code>	Insert or update.
MERGE	<code>MERGE INTO target USING source ON (...) WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...;</code>	Synchronize datasets.
Ignore duplicate — PostgreSQL	<code>ON CONFLICT DO NOTHING</code>	Skip conflicting insert.
Ignore duplicate — MySQL	<code>INSERT IGNORE ...</code>	Skip certain insert errors; understand which errors are suppressed.

39. DATA LOADING / BULK IMPORT

Operation	Query / command	When / Where to use
PostgreSQL COPY	<code>COPY employee FROM '/path/employee.csv' CSV HEADER;</code>	Bulk-load CSV where server permissions allow.
PostgreSQL psql \copy	<code>\copy employee FROM 'employee.csv' CSV HEADER</code>	Client-side CSV loading.
MySQL LOAD DATA	<code>LOAD DATA INFILE 'employee.csv' INTO TABLE employee ...;</code>	Bulk-load text/CSV data.
SQL Server BULK INSERT	<code>BULK INSERT employee FROM 'employee.csv' WITH (...);</code>	Bulk import in SQL Server.
SQL Server OPENROWSET	<code>SELECT * FROM OPENROWSET(...)</code>	Query/load external data where configured.

BigQuery load	<code>bq load ...</code>	Bulk-load files into BigQuery; command-line/API syntax is platform-specific.
Snowflake COPY INTO	<code>COPY INTO target FROM @stage;</code>	Load staged files into Snowflake.
Redshift COPY	<code>COPY target FROM 's3://bucket/file.csv' ...;</code>	Load data from object storage.

40. EXTERNAL / CLOUD DATA

Platform feature	Query / command	When / Where to use
BigQuery external table	<code>CREATE EXTERNAL TABLE ... OPTIONS (...)</code>	Query external files without fully loading them.
Snowflake external stage	<code>CREATE STAGE my_stage ...;</code>	Define external/internal file staging.
Snowflake COPY	<code>COPY INTO table FROM @stage;</code>	Load staged data.
Redshift Spectrum	<code>CREATE EXTERNAL SCHEMA ...;</code>	Query external data in supported catalogs.
Databricks SQL	<code>CREATE TABLE ... USING DELTA;</code>	Create tables over Delta Lake.
Delta table	<code>CREATE TABLE t USING DELTA AS SELECT ...;</code>	Lakehouse table format in Databricks.

41. SQL SERVER-SPECIFIC PATTERNS

Feature	Query	When / Where to use
TOP	<code>SELECT TOP 10 * FROM employee;</code>	Limit rows.
IDENTITY	<code>id INT IDENTITY(1,1) PRIMARY KEY</code>	Auto-generated identity column.
OUTPUT	<code>INSERT ... OUTPUT inserted.id ...</code>	Return affected rows/IDs from DML.

APPLY	CROSS APPLY (...) / OUTER APPLY (...)	Invoke table-valued expressions per outer row.
PIVOT	PIVOT (SUM(amount) FOR month IN (...))	Pivot rows to columns.
UNPIVOT	UNPIVOT (amount FOR month IN (...))	Unpivot columns to rows.
MERGE	MERGE ...	Synchronize source and target; assess concurrency/behavior carefully.
TRY_CONVERT	TRY_CONVERT(INT,value)	Safe conversion.
DATEADD	DATEADD(day,7,date_col)	Date arithmetic.
DATEDIFF	DATEDIFF(day,start_date,end_date)	Date difference.

42. POSTGRESQL-SPECIFIC PATTERNS

Feature	Query	When / Where to use
ILIKE	WHERE name ILIKE 'y%'	Case-insensitive pattern matching.
RETURNING	INSERT INTO employee(...) VALUES(...) RETURNING emp_id;	Return affected rows/IDs.
ON CONFLICT	INSERT ... ON CONFLICT(id) DO UPDATE ...	UPSERT.
DISTINCT ON	SELECT DISTINCT ON (dept_id) * FROM employee ORDER BY dept_id,salary DESC;	One representative row per key.
FILTER	COUNT(*) FILTER (WHERE active)	Conditional aggregation.
LATERAL	FROM customer c CROSS JOIN LATERAL (...) x	Correlated table expression.
JSONB	data->>'name'	Query JSONB.

ARRAY	<code>WHERE 'SQL'=ANY(tags)</code>	Query arrays.
GENERATE_SERIES	<code>SELECT * FROM generate_series(1,10);</code>	Generate sequences.
DATE_TRUNC	<code>DATE_TRUNC('month',order_date)</code>	Bucket timestamps.

43. MYSQL / MARIADB-SPECIFIC PATTERNS

Feature	Query	When / Where to use
LIMIT	<code>SELECT * FROM employee LIMIT 10;</code>	Limit rows.
AUTO_INCREMENT	<code>id INT AUTO_INCREMENT PRIMARY KEY</code>	Generate IDs.
ON DUPLICATE KEY UPDATE	<code>INSERT ... ON DUPLICATE KEY UPDATE ...</code>	UPSERT.
INSERT IGNORE	<code>INSERT IGNORE INTO ...</code>	Skip certain insert errors/conflicts.
GROUP_CONCAT	<code>GROUP_CONCAT(name SEPARATOR ' ',')</code>	Aggregate strings.
JSON_EXTRACT	<code>JSON_EXTRACT(data, '\$.name')</code>	Query JSON.
FIND_IN_SET	<code>FIND_IN_SET('SQL',tags)</code>	Search comma-separated lists; normalized tables are usually preferable.
REGEXP	<code>WHERE name REGEXP '^A'</code>	Regular-expression matching where supported.
LOAD DATA	<code>LOAD DATA INFILE ...</code>	Bulk import.
SHOW	<code>SHOW TABLES;</code>	Metadata inspection.

44. ORACLE-SPECIFIC PATTERNS

Feature	Query	When / Where to use
---------	-------	---------------------

ROWNUM	<code>SELECT * FROM employee WHERE ROWNUM <= 10;</code>	Legacy row limiting.
FETCH FIRST	<code>SELECT * FROM employee FETCH FIRST 10 ROWS ONLY;</code>	Modern Oracle row limiting.
ROWID	<code>SELECT ROWID, employee.* FROM employee;</code>	Oracle physical row locator; specialized use.
CONNECT BY	<code>SELECT ... CONNECT BY PRIOR emp_id=manager_id START WITH manager_id IS NULL;</code>	Hierarchical queries.
MERGE	<code>MERGE INTO target USING source ON (...)</code> <code>...</code>	Synchronization/upsert.
NVL	<code>NVL(phone, 'None')</code>	Replace NULL.
LISTAGG	<code>LISTAGG(name, ', ') WITHIN GROUP (ORDER BY name)</code>	String aggregation.
SYSDATE	<code>SELECT SYSDATE FROM dual;</code>	Current server date/time.
DUAL	<code>SELECT 1 FROM dual;</code>	Oracle one-row dummy table.

45. SQLITE-SPECIFIC PATTERNS

Feature	Query	When / Where to use
LIMIT/OFFSET	<code>SELECT * FROM employee LIMIT 10 OFFSET 20;</code>	Pagination.
AUTOINCREMENT	<code>id INTEGER PRIMARY KEY AUTOINCREMENT</code>	Generate IDs; SQLite-specific behavior.
RETURNING	<code>INSERT ... RETURNING id;</code>	Return affected rows in supported SQLite versions.
UPSERT	<code>INSERT ... ON CONFLICT(id) DO UPDATE SET ...;</code>	Insert/update.
strftime	<code>strftime('%Y-%m', order_date)</code>	Date formatting/extraction.

json_extract	<code>json_extract(data, '\$.name')</code>	JSON1 extension.
PRAGMA	<code>PRAGMA foreign_keys = ON;</code>	SQLite configuration/metadata.

46. BIGQUERY-SPECIFIC PATTERNS

Feature	Query	When / Where to use
QUALIFY	<code>SELECT ... QUALIFY ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)=1;</code>	Filter window-function results without an extra subquery.
UNNEST	<code>SELECT item FROM t, UNNEST(items) item;</code>	Flatten arrays/repeated fields.
SAFE_CAST	<code>SAFE_CAST(value AS INT64)</code>	Return NULL rather than fail on invalid conversion.
ARRAY_AGG	<code>ARRAY_AGG(name)</code>	Aggregate values into an array.
STRUCT	<code>STRUCT(name,salary)</code>	Build structured values.
PARTITION BY table	<code>CREATE TABLE t PARTITION BY DATE(order_date) AS SELECT ...;</code>	Partition analytical tables.
CLUSTER BY table	<code>CREATE TABLE t PARTITION BY DATE(order_date) CLUSTER BY customer_id AS SELECT ...;</code>	Improve pruning for common filters.

47. SNOWFLAKE-SPECIFIC PATTERNS

Feature	Query	When / Where to use
QUALIFY	<code>SELECT ... QUALIFY ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)=1;</code>	Filter window results directly.
FLATTEN	<code>SELECT f.value FROM table, LATERAL FLATTEN(input=>data) f;</code>	Flatten semi-structured data.

VARIANT	<code>data:name::STRING</code>	Query semi-structured data.
COPY INTO	<code>COPY INTO target FROM @stage;</code>	Load staged data.
CREATE STAGE	<code>CREATE STAGE my_stage;</code>	Create a data-loading stage.
CLUSTER BY	<code>CREATE TABLE t (...) CLUSTER BY (date_col, customer_id);</code>	Define clustering keys for pruning.
TIME TRAVEL	<code>SELECT * FROM table AT (OFFSET => -60*5);</code>	Query historical state where enabled.

48. REDSHIFT-SPECIFIC PATTERNS

Feature	Query	When / Where to use
DISTKEY	<code>CREATE TABLE t (...) DISTKEY(customer_id);</code>	Distribute rows by key in Redshift.
SORTKEY	<code>CREATE TABLE t (...) SORTKEY(order_date);</code>	Optimize scans/sorts using sort keys.
COPY	<code>COPY target FROM 's3://bucket/file.csv' ...;</code>	Bulk-load from object storage.
UNLOAD	<code>UNLOAD ('SELECT ...') TO 's3://bucket/out/' ...;</code>	Export query results.
VACUUM	<code>VACUUM table_name;</code>	Maintain table sort/space where required.
ANALYZE	<code>ANALYZE table_name;</code>	Update optimizer statistics.

49. DATABRICKS / LAKEHOUSE SQL

Feature	Query	When / Where to use
Delta table	<code>CREATE TABLE t USING DELTA AS SELECT ...;</code>	Create a Delta Lake table.

MERGE INTO	<code>MERGE INTO target USING source ON ... WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...;</code>	Incremental upsert/CDC.
OPTIMIZE	<code>OPTIMIZE table_name;</code>	Compact Delta files.
ZORDER	<code>OPTIMIZE table_name ZORDER BY (customer_id);</code>	Improve data skipping for common filters.
VACUUM	<code>VACUUM table_name;</code>	Remove old files according to retention rules.
LATERAL VIEW	<code>SELECT ... FROM t LATERAL VIEW explode(items) x AS item;</code>	Explode arrays/maps in Spark SQL syntax.
explode	<code>SELECT explode(items) FROM t;</code>	Convert arrays into rows.

50. REGULAR EXPRESSIONS

Feature	Query	When / Where to use
PostgreSQL regex	<code>WHERE name ~ '^A'</code>	Regex matching.
PostgreSQL case-insensitive regex	<code>WHERE name ~* '^a'</code>	Case-insensitive regex.
MySQL REGEXP	<code>WHERE name REGEXP '^A'</code>	Regex matching.
Oracle REGEXP_LIKE	<code>WHERE REGEXP_LIKE(name, '^A')</code>	Regex predicate.
Oracle REGEXP_SUBSTR	<code>REGEXP_SUBSTR(text, pattern)</code>	Extract regex match.
Oracle REGEXP_REPLACE	<code>REGEXP_REPLACE(text, pattern, replacement)</code>	Regex replacement.
BigQuery REGEXP_CONTAINS	<code>REGEXP_CONTAINS(name, r'^A')</code>	Regex predicate.

BigQuery REGEXP_EXTRACT	REGEXP_EXTRACT(text, r'pattern')	Extract regex match.
----------------------------	----------------------------------	----------------------

51. SECURITY: PARAMETERIZED SQL / SQL INJECTION

Pattern	Query	When / Where to use
Unsafe concatenation	<code>SELECT * FROM users WHERE email=' + input + ';</code>	Do not use with untrusted input.
Parameterized query	<code>SELECT * FROM users WHERE email = ?;</code>	Use application parameters instead of string concatenation.
Named parameter	<code>SELECT * FROM users WHERE email = :email;</code>	Use when the application driver supports named parameters.
Prepared statement	<code>PREPARE stmt FROM 'SELECT ... WHERE id=?';</code>	Prepare reusable SQL where supported.

52. COMMON INTERVIEW QUERY PATTERNS

Problem	Typical solution	When / Where to use
Second highest salary	<code>DENSE_RANK() OVER (ORDER BY salary DESC) then WHERE rnk=2</code>	Second distinct maximum.
Nth highest salary	<code>DENSE_RANK() OVER (ORDER BY salary DESC) then WHERE rnk=N</code>	Nth distinct maximum.
Duplicate values	<code>GROUP BY key HAVING COUNT(*)>1</code>	Find duplicates.
Remove duplicates	<code>ROW_NUMBER() OVER (PARTITION BY key ORDER BY id)</code>	Keep one copy and remove/filter extras.
Employees above average	<code>WHERE salary > (SELECT AVG(salary) FROM employee)</code>	Compare against overall average.
Employees above department average	<code>salary > AVG(salary) OVER (PARTITION BY dept_id)</code>	Compare against group average.

Highest salary per department	<code>ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)</code>	Top row in every group.
Employees with no project	<code>WHERE NOT EXISTS (...)</code>	Find missing relationships.
Departments with no employees	<code>LEFT JOIN ... WHERE employee.id IS NULL</code>	Find unmatched parents.
Consecutive records	<code>LAG() / LEAD()</code>	Detect adjacent events/dates.
Running total	<code>SUM(x) OVER (ORDER BY date)</code>	Cumulative metric.
Month-over-month	<code>LAG(metric) OVER (ORDER BY month)</code>	Compare periods.
Gaps and islands	Window functions + grouping	Find consecutive sequences/groups.
Median	<code>PERCENTILE_CONT(0.5) ...</code>	Calculate median where supported.
Top N per group	<code>ROW_NUMBER() OVER (PARTITION BY group ORDER BY metric DESC)</code>	Top records in every group.

53. QUERY EXECUTION ORDER

SQL is written in one order but logically processed approximately in this order:

```
FROM
JOIN
ON
WHERE
GROUP BY
HAVING
SELECT
DISTINCT
ORDER BY
LIMIT / OFFSET / FETCH
```

Example:

```
SELECT department_id,
       AVG(salary) AS avg_salary
FROM employee
WHERE salary > 50000
GROUP BY department_id
HAVING AVG(salary) > 60000
ORDER BY avg_salary DESC
FETCH FIRST 5 ROWS ONLY;
```

54. WHERE vs HAVING

WHERE	HAVING
Filters rows	Filters groups
Usually before aggregation	After grouping/aggregation
WHERE salary > 50000	HAVING AVG(salary) > 60000

55. GROUP BY vs PARTITION BY

GROUP BY	PARTITION BY
Collapses rows into groups	Keeps individual rows
Used with aggregate queries	Used inside window functions
One result row per group is typical	One result row per input row is typical
GROUP BY dept_id	AVG(salary) OVER (PARTITION BY dept_id)

56. DELETE vs TRUNCATE vs DROP

Operation	Rows removed	Table remains	WHERE	Typical purpose
-----------	--------------	---------------	-------	-----------------

DELETE	Yes	Yes	Yes	Remove selected rows
TRUNCATE	All	Yes	No	Empty table
DROP	All	No	No	Remove object

Exact transaction, identity, trigger, locking, and logging behavior varies by DBMS.

57. UNION vs JOIN

JOIN

→ combines related columns horizontally

UNION

→ combines compatible result rows vertically

58. ROW_NUMBER vs RANK vs DENSE_RANK

For:

100000

100000

90000

80000

ROW_NUMBER → 1, 2, 3, 4

RANK → 1, 1, 3, 4

DENSE_RANK → 1, 1, 2, 3

59. SQL DIALECT MAP

DBMS/platform	Dialect / language	Examples of distinctive features
MySQL	MySQL SQL	LIMIT , AUTO_INCREMENT , ON DUPLICATE KEY , GROUP_CONCAT
MariaDB	MariaDB SQL	MySQL-compatible syntax plus MariaDB features
PostgreSQL	PostgreSQL SQL / PL/pgSQL	RETURNING , ON CONFLICT , ILIKE , arrays, JSONB, LATERAL
SQL Server	T-SQL	TOP , OUTPUT , APPLY , PIVOT , TRY_CONVERT
Oracle	Oracle SQL / PL/SQL	CONNECT BY , ROWNUM , MERGE , LISTAGG , packages
SQLite	SQLite SQL	lightweight embedded SQL, PRAGMA , JSON1, RETURNING
BigQuery	GoogleSQL	QUALIFY , UNNEST , arrays/structs, partitioned analytical tables
Snowflake	Snowflake SQL	QUALIFY , FLATTEN , VARIANT , stages, COPY INTO
Redshift	Redshift SQL	DISTKEY , SORTKEY , COPY , UNLOAD
Databricks	Databricks SQL / Spark SQL	Delta, MERGE , OPTIMIZE , ZORDER , explode

60. MASTER SQL LEARNING ORDER

1. SELECT
2. WHERE
3. AND / OR / NOT
4. IN / NOT IN
5. BETWEEN
6. LIKE
7. NULL
8. ORDER BY
9. LIMIT / OFFSET / FETCH / TOP
10. DISTINCT
11. Aggregate functions
12. GROUP BY
13. HAVING
14. INNER JOIN
15. LEFT JOIN
16. RIGHT / FULL / CROSS / SELF JOIN
17. UNION / UNION ALL
18. Subqueries
19. EXISTS / NOT EXISTS
20. CASE
21. COALESCE / NULLIF
22. CTE
23. Recursive CTE
24. Window functions
25. PARTITION BY
26. ROW_NUMBER / RANK / DENSE_RANK
27. LAG / LEAD
28. Window frames
29. GROUPING SETS / ROLLUP / CUBE
30. Views
31. Materialized views
32. Indexes
33. EXPLAIN
34. Transactions
35. Isolation levels
36. Locks
37. Constraints
38. Procedures / functions

- 39. Triggers
- 40. JSON
- 41. Arrays
- 42. Full-text search
- 43. Partitioned tables
- 44. Bulk loading
- 45. Security / roles
- 46. Metadata/catalogs
- 47. DBMS-specific syntax
- 48. Cloud data warehouse SQL
- 49. Query optimization
- 50. Advanced analytical SQL

61. FINAL CHEAT SHEET

Need data?

→ SELECT

Need filtering?

→ WHERE

Need unique results?

→ DISTINCT

Need sorting?

→ ORDER BY

Need a range?

→ BETWEEN

Need a list?

→ IN

Need pattern matching?

→ LIKE / REGEXP

Need NULL handling?

→ IS NULL / COALESCE / NULLIF

Need totals?

→ SUM / COUNT / AVG / MIN / MAX

Need groups?

→ GROUP BY

Need filter groups?

→ HAVING

Need another table?

→ JOIN

Need no matching row?

→ NOT EXISTS / LEFT JOIN ... IS NULL

Need combine query results?

→ UNION / INTERSECT / EXCEPT

Need a reusable query step?

→ CTE

Need recursion?

→ RECURSIVE CTE

Need rank?

→ RANK / DENSE_RANK / ROW_NUMBER

Need calculation within each group?

→ PARTITION BY

Need previous/next row?

→ LAG / LEAD

Need running total?

→ SUM() OVER(...)

Need top N per group?

→ ROW_NUMBER() OVER(PARTITION BY ...)

Need physical large-table partitioning?

→ RANGE / LIST / HASH partitioning

Need add data?

→ INSERT

Need change data?

→ UPDATE

Need remove selected rows?

→ DELETE

Need empty a table?

→ TRUNCATE

Need remove a table?

→ DROP

Need insert-or-update?

→ UPSERT / MERGE

Need transactions?

→ BEGIN / COMMIT / ROLLBACK / SAVEPOINT

Need permissions?

→ GRANT / REVOKE / ROLE

Need performance?

→ INDEX / EXPLAIN / ANALYZE

Need reusable database logic?

→ VIEW / FUNCTION / PROCEDURE / TRIGGER

Need semi-structured data?

→ JSON / ARRAY / UNNEST / FLATTEN

Need analytics?

→ WINDOW FUNCTIONS / GROUPING SETS / ROLLUP / CUBE

Need cloud data?

→ BigQuery / Snowflake / Redshift / Databricks-specific SQL

62. Important Limitation

This is a **global practical SQL reference**, not a literal dump of every SQL statement ever implemented by every database vendor.

A truly exhaustive vendor reference would include thousands of version-specific features, administrative commands, proprietary procedural statements, extensions, optimizer hints, system catalog APIs, and cloud-specific functions.

The correct way to use this document is:

Standard SQL concepts first → major DBMS syntax → platform-specific extensions → version-specific documentation.